

Automated Warehouse Robot Controller

Multi-Agent Pathfinding with Cooperative Search Algorithms

Project 9 — Technical Documentation

Course: Introduction to Artificial Intelligence
Topic: AI Search Algorithms — MAPF & Cooperative Search
Institution: École Nationale Supérieure d'Informatique (ENSIA)
Academic Year: 2025–2026

Algorithms Covered

Independent A*	Baseline (with conflict detection)
Cooperative A*	Sequential priority planning
Hill Climbing	Local search optimizer
Conflict-Based Search (CBS)	Optimal two-level MAPF solver
Space-Time A*	Core low-level planner

Contents

1	Problem Overview	3
1.1	Motivation	3
1.2	Problem Formulation	3
1.3	Constraints	3
1.3.1	Hard Constraints (must never be violated)	4
1.3.2	Soft Constraints (optimize)	4
1.4	Objective Functions	4
2	System Architecture & Data Structures	5
2.1	Overview	5
2.2	Class: <code>WarehouseGrid</code>	6
2.2.1	Purpose	6
2.2.2	Representation	6
2.2.3	Action Space	6
2.2.4	Key Methods	6
2.2.5	Pre-defined Maps	6
2.3	Class: <code>ReservationTable</code>	7
2.3.1	Purpose & Motivation	7
2.3.2	Vertex Reservations	8
2.3.3	Edge Reservations (Swap Prevention)	8
2.3.4	Tail Reservations	8
2.3.5	Complexity	8
2.4	Classes: <code>Robot</code> and <code>WarehouseTask</code>	9
2.4.1	<code>Robot</code>	9
2.4.2	<code>WarehouseTask</code>	9
3	Core Planner: Space-Time A*	11
3.1	From 2D A* to Space-Time A*	11
3.2	State Space Definition	11
3.3	Heuristic Function	11
3.4	Successor Generation	12
3.5	Full Space-Time A* Algorithm	13
3.6	Path Reconstruction	13
3.7	Complexity Analysis	14
4	Algorithm 1: Independent A* (Baseline)	15
4.1	Concept	15
4.2	Algorithm	15
4.3	Why It Fails in Dense Environments	16

4.4	Use Case	16
5	Algorithm 2: Cooperative A* (CA*)	17
5.1	Concept	17
5.2	Algorithm	17
5.3	Detailed Walk-through	17
5.4	Properties	18
5.5	Priority Ordering Strategies	18
5.6	Head-on Collision Example	18
6	Algorithm 3: Hill Climbing Optimizer	19
6.1	Motivation	19
6.2	Concept	19
6.3	Algorithm	20
6.4	Why Conflict-Freedom is Preserved	20
6.5	Convergence & Limitations	20
7	Algorithm 4: Conflict-Based Search (CBS)	21
7.1	Overview	21
7.2	Key Data Structures	21
7.2.1	CBSConstraint	21
7.2.2	CBSNode	21
7.3	High-Level Algorithm	22
7.4	Low-Level Algorithm	22
7.5	Optimality Proof Sketch	22
7.6	Complexity	23
8	Visualization System	24
8.1	Animated Simulation	24
8.2	Congestion Heatmap	24
8.3	Static Snapshot	24
9	Comparative Evaluation	25
9.1	Experiment 1: Success Rate	25
9.2	Experiment 2: Fleet Size Scaling	25
9.3	Experiment 3: Head-on Collision	25
10	Summary & Conclusions	26
10.1	Algorithm Comparison Table	26
10.2	Key Findings	26
10.3	Limitations & Future Work	27
A	Class & Function Reference	28
B	Conflict Type Reference	29
C	Glossary	30

Chapter 1

Problem Overview

1.1 Motivation

Modern warehouses such as Amazon Fulfillment Centers operate fleets of hundreds of autonomous mobile robots (AMRs) simultaneously. These robots must navigate a dense, shared environment — retrieving shelves, delivering items to packing stations, and returning — all without colliding with one another or reaching a deadlock state from which no robot can escape.

This project designs and implements a complete **Multi-Agent Pathfinding (MAPF)** system that coordinates N warehouse robots on a 2D grid map, comparing four distinct AI search strategies in terms of correctness, efficiency, and scalability.

1.2 Problem Formulation

MAPF Problem (Formal Definition)

Given:

- A grid graph $G = (V, E)$ where V is the set of passable cells and E connects adjacent cells
- N agents, each with a start position $s_i \in V$ and goal position $g_i \in V$

Find: A set of paths $\pi = \{\pi_1, \pi_2, \dots, \pi_N\}$ such that:

1. Each π_i is a valid path from s_i to g_i in G
2. No two paths produce a *vertex conflict*: $\pi_i(t) \neq \pi_j(t)$ for all $i \neq j$ and all timesteps t
3. No two paths produce an *edge conflict*: it is not the case that $\pi_i(t) = \pi_j(t-1)$ and $\pi_i(t-1) = \pi_j(t)$ simultaneously

Minimize: The **makespan** $\max_i |\pi_i|$ or the **flowtime** $\sum_i |\pi_i|$.

1.3 Constraints

1.3.1 Hard Constraints (must never be violated)

Important

1. **Vertex conflict:** No two robots occupy the same cell (r, c) at the same timestep t .
2. **Edge conflict (swap):** Robot A moves $u \rightarrow v$ while robot B moves $v \rightarrow u$ in the same step — forbidden even if neither occupies the same cell simultaneously.
3. **Obstacle constraint:** No robot may enter a cell marked as an obstacle ($\text{grid}[r][c] = 1$).

1.3.2 Soft Constraints (optimize)

- Minimize **waiting time** (idle steps where the robot does not move forward).
- Maintain a **safety buffer** between robots where possible.
- Minimize **path crossing** in high-traffic corridors.

1.4 Objective Functions

Two complementary metrics measure solution quality:

$$\text{Makespan} = \max_{i=1}^N |\pi_i| \quad (1.1)$$

$$\text{Flowtime} = \sum_{i=1}^N |\pi_i| \quad (1.2)$$

Makespan measures the total wall-clock time until the last robot finishes. Flowtime measures total robot-hours consumed. In practice, both metrics are relevant: makespan matters for throughput, flowtime matters for energy efficiency.

Chapter 2

System Architecture & Data Structures

2.1 Overview

The system is organized into five layers:

1. **Grid World** — environment representation
2. **Reservation Table** — shared collision-prevention structure
3. **Space-Time A*** — low-level single-robot planner
4. **Multi-Agent Planners** — four high-level strategies
5. **Simulation & Visualization** — animation and analysis

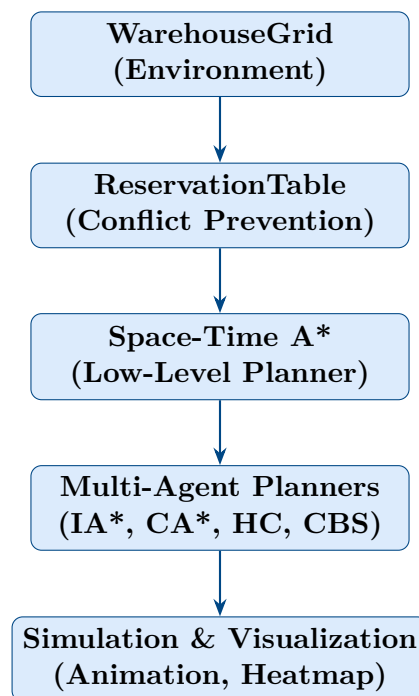


Figure 2.1: System architecture — five-layer design

2.2 Class: WarehouseGrid

2.2.1 Purpose

`WarehouseGrid` encodes the physical environment as a 2D integer matrix. It provides the complete graph structure (neighbors, validity checks) that all search algorithms operate on.

2.2.2 Representation

$$\text{grid}[r][c] = \begin{cases} 0 & \text{free cell (robot may enter)} \\ 1 & \text{obstacle (wall or fixed shelf)} \end{cases} \quad (2.1)$$

The grid is indexed as $r \in \{0, \dots, R-1\}$ (rows, top to bottom) and $c \in \{0, \dots, C-1\}$ (columns, left to right).

2.2.3 Action Space

At each timestep, a robot may take one of five actions:

Action	Meaning	Δr	Δc
UP	Move one row up	-1	0
DOWN	Move one row down	+1	0
LEFT	Move one col left	0	-1
RIGHT	Move one col right	0	+1
WAIT	Stay in place	0	0

Table 2.1: Robot action space — 4-directional movement plus wait

2.2.4 Key Methods

- `is_valid(r, c)` — Returns `True` iff (r, c) is inside the grid and not an obstacle. Used by every search to prune invalid moves in $O(1)$.
- `get_neighbors(r, c)` — Returns all valid successor cells (including WAIT) as a list of `(new_r, new_c, action_name)` triples.
- `free_cells()` — Returns a flat list of all passable cells, used by the task generator to place robots randomly.
- `visualize(...)` — Renders the grid using Matplotlib with color coding: obstacles in dark gray, free cells in light gray, robot starts as colored circles, goals as dashed colored rectangles.

2.2.5 Pre-defined Maps

Two warehouse layouts are provided:

Map	Size	Free cells	Shelf rows	Use case
Small	8×10	48	2	Quick testing
Medium	12×16	96	3	Realistic scale

Table 2.2: Pre-defined warehouse maps

Both maps follow a real warehouse pattern: horizontal shelf blocks separated by clear aisles, with an open zone at the bottom representing packing stations.

```

1 def create_small_warehouse():
2     grid = [
3         [0, 0, 0, 0, 0, 0, 0, 0, 0, 0], # Row 0: top aisle
4         [0, 1, 1, 0, 1, 1, 0, 1, 1, 0], # Row 1: shelf
5         [0, 1, 1, 0, 1, 1, 0, 1, 1, 0], # Row 2: shelf
6         [0, 0, 0, 0, 0, 0, 0, 0, 0, 0], # Row 3: middle
7         [0, 1, 1, 0, 1, 1, 0, 1, 1, 0], # Row 4: shelf
8         [0, 1, 1, 0, 1, 1, 0, 1, 1, 0], # Row 5: shelf
9         [0, 0, 0, 0, 0, 0, 0, 0, 0, 0], # Row 6: middle
10        [0, 0, 0, 0, 0, 0, 0, 0, 0, 0], # Row 7: packing
11        ]
12    return WarehouseGrid(grid)

```

Listing 2.1: Small warehouse grid definition (8×10)

2.3 Class: ReservationTable

2.3.1 Purpose & Motivation

The **ReservationTable** is the central data structure for collision prevention in all cooperative planners. Instead of checking collisions reactively (after the fact), it allows planning robots to proactively *avoid* cells that other robots will occupy at specific future times.

Reservation Table (Space-Time Occupancy Grid)

A reservation table is a set of triples:

$$\mathcal{R}_v = \{(r, c, t) \mid \text{some robot occupies cell } (r, c) \text{ at timestep } t\}$$

and a set of quintuples:

$$\mathcal{R}_e = \{(r_1, c_1, r_2, c_2, t) \mid \text{some robot traverses edge } (r_1, c_1) \rightarrow (r_2, c_2) \text{ at time } t\}$$

2.3.2 Vertex Reservations

When robot i plans path $\pi_i = [(r_0, c_0), (r_1, c_1), \dots, (r_T, c_T)]$, for each timestep t , the triple (r_t, c_t, t) is added to $\mathcal{R}_v$.

Any subsequent robot's planner queries:

$$\text{is_vertex_free}(r, c, t) \iff (r, c, t) \notin \mathcal{R}_v$$

This enforces the hard vertex-conflict constraint.

2.3.3 Edge Reservations (Swap Prevention)

When robot i moves from (r_1, c_1) to (r_2, c_2) at time t , two edge reservations are added:

$$(r_1, c_1, r_2, c_2, t) \in \mathcal{R}_e \quad (\text{actual traversal}) \quad (2.2)$$

$$(r_2, c_2, r_1, c_1, t) \in \mathcal{R}_e \quad (\text{reverse blocked}) \quad (2.3)$$

The reverse reservation blocks any other robot from performing the opposite traversal at the same time — preventing head-on swap conflicts.

2.3.4 Tail Reservations

After a robot reaches its goal, it does not disappear. To prevent other robots from walking through a stationary waiting robot, 50 extra vertex reservations are added at the final position:

$$(r_T, c_T, T + k) \in \mathcal{R}_v \quad \text{for } k = 1, 2, \dots, 50$$

2.3.5 Complexity

Operation	Structure	Complexity
Reserve path of length T	Python <code>set.add()</code>	$O(T)$
Query vertex (r, c, t)	Python <code>set.__contains__</code>	$O(1)$ avg
Query edge (r_1, c_1, r_2, c_2, t)	Python <code>set.__contains__</code>	$O(1)$ avg
Clear table	<code>set.clear()</code>	$O(\mathcal{R})$

Table 2.3: Reservation table operation complexities

```

1 def reserve_path(self, path, robot_id):
2     for t, (r, c) in enumerate(path):
3         self.vertex_reservations.add((r, c, t)) # Vertex
           reservation
4         if t > 0:
5             prev_r, prev_c = path[t - 1]
6             # Block this traversal AND the reverse (swap
           prevention)
7             self.edge_reservations.add((prev_r, prev_c, r, c,
           t))

```

```

8         self.edge_reservations.add((r, c, prev_r, prev_c,
9         t))
10        # Tail reservations      robot stays at goal
11        final_r, final_c = path[-1]
12        for extra_t in range(len(path), len(path) + 50):
13            self.vertex_reservations.add((final_r, final_c,
14            extra_t))

```

Listing 2.2: Core reservation table methods

2.4 Classes: Robot and WarehouseTask

2.4.1 Robot

Each robot is characterized by:

- **robot_id** — unique integer identifier
- **start** — (r, c) initial grid position
- **goal** — (r, c) target grid position
- **path** — list of (r, c) positions at each timestep (set by the planner)
- **color** — unique RGB color for visualization

The method `position_at(t)` safely handles robots that have already reached their goal: if $t \geq |\pi_i|$, the last position is returned (robot waits at goal indefinitely).

2.4.2 WarehouseTask

`WarehouseTask` encapsulates a complete MAPF problem instance. Its key responsibility beyond storage is providing solution-quality metrics and post-hoc conflict detection:

```

1  def check_conflicts(self):
2      conflicts = []
3      for t in range(self.makespan() + 1):
4          # --- Vertex conflict check ---
5          positions = {}
6          for robot in self.robots:
7              pos = robot.position_at(t)
8              if pos in positions:
9                  conflicts.append({'type': 'vertex', 'time': t
10
11                                     'position': pos,
12                                     'robots': (positions[pos],
13
14                                     robot.robot_id)})
15          positions[pos] = robot.robot_id
16          # --- Edge (swap) conflict check ---
17          if t > 0:
18              moves = {}
19              for robot in self.robots:
20                  prev = robot.position_at(t - 1)
21                  curr = robot.position_at(t)
22                  moves[(prev, curr)] = robot.robot_id

```

```
20         for robot in self.robots:
21             prev = robot.position_at(t - 1)
22             curr = robot.position_at(t)
23             if (curr, prev) in moves and moves[(curr, prev
24 )] != robot.robot_id:
25                 conflicts.append({'type': 'edge (swap)',
26 'time': t, ...})
27         return conflicts
```

Listing 2.3: Conflict detection logic in WarehouseTask

Chapter 3

Core Planner: Space-Time A*

3.1 From 2D A* to Space-Time A*

Classical A* searches in a 2D state space (r, c) . For a single robot in a static environment this is optimal and complete. However, in a dynamic multi-robot setting the obstacles (other robots) are not static — they move over time.

Space-Time A* extends the state space to three dimensions:

$$\text{state} = (r, c, t)$$

where t is the current timestep. A state (r, c, t) is *unavailable* if the reservation table has reserved (r, c, t) .

Key Insight

By searching in the (r, c, t) state space, the planner can discover paths that include **WAIT actions** at the optimal moment — pausing when a corridor is temporarily blocked and proceeding as soon as it clears. This naturally resolves most inter-robot conflicts without explicit conflict detection.

3.2 State Space Definition

$$S = \{(r, c, t) \mid (r, c) \in V, t \in \{0, 1, \dots, T_{\max}\}\} \quad (3.1)$$

$$s_0 = (r_{\text{start}}, c_{\text{start}}, 0) \quad (3.2)$$

$$S_G = \{(r_{\text{goal}}, c_{\text{goal}}, t) \mid t \geq 0\} \quad (3.3)$$

3.3 Heuristic Function

The heuristic is the **Manhattan distance** from the current position to the goal, ignoring the time dimension:

$$h(r, c, t) = |r - r_G| + |c - c_G|$$

Proposition 1. *The Manhattan distance heuristic is admissible for 4-directional grid movement: it never overestimates the true cost to reach the goal, because each step can reduce the Manhattan distance by at most 1.*

Proof. Each move changes (r, c) by exactly one unit in one direction. Therefore, the minimum number of steps to reach (r_G, c_G) from (r, c) is at least $|r - r_G| + |c - c_G|$. The WAIT action does not reduce Manhattan distance at all. Hence $h \leq h^*$ everywhere. $\square$ $\square$

3.4 Successor Generation

From state (r, c, t) , successors are generated for each of the 5 actions:

Algorithm 1 Space-Time A* Successor Generation

```

1: for each action  $a \in \{\text{UP, DOWN, LEFT, RIGHT, WAIT}\}$  do
2:    $(nr, nc) \leftarrow (r + \Delta r_a, c + \Delta c_a)$ 
3:    $t' \leftarrow t + 1$ 
4:   if grid.is_valid(nr, nc) then
5:     if not reservation.is_vertex_free(nr, nc, t') then
6:       continue  $\triangleright$  Vertex conflict — skip
7:     end if
8:     if not reservation.is_edge_free(r, c, nr, nc, t') then
9:       continue  $\triangleright$  Edge/swap conflict — skip
10:    end if
11:    Create child node  $(nr, nc, t', g + 1, h(nr, nc, t'), \text{parent} = \text{current})$ 
12:    Push child onto open heap with priority  $f = g + 1 + h$ 
13:  end if
14: end for

```

3.5 Full Space-Time A* Algorithm

Algorithm 2 Space-Time A* (Single Robot)

Require: Grid G , start (r_s, c_s) , goal (r_g, c_g) , reservation table $\mathcal{R}$, $T_{\max}$

Ensure: Collision-free path or None

```

1: open  $\leftarrow \{(f_0, \text{node}(r_s, c_s, 0, g = 0, h = h_0))\}$ 
2: closed  $\leftarrow \emptyset$ 
3: while open  $\neq \emptyset$  do
4:    $(f, n) \leftarrow \text{HEAPPop}(\text{open})$ 
5:   if  $(n.r, n.c, n.t) \in \text{closed}$  then continue
6:   end if
7:   closed  $\leftarrow \text{closed} \cup \{(n.r, n.c, n.t)\}$ 
8:   if  $(n.r, n.c) = (r_g, c_g)$  then
9:     return RECONSTRUCTPATH( $n$ )
10:  end if
11:  if  $n.t \geq T_{\max}$  then continue
12:  end if
13:  for each valid successor  $(nr, nc, t')$  do
14:    if  $(nr, nc, t') \notin \text{closed}$  then
15:       $g' \leftarrow n.g + 1$ 
16:       $h' \leftarrow |nr - r_g| + |nc - c_g|$ 
17:      HEAPPUSH(open,  $(g' + h', \text{node}(nr, nc, t', g', h', n))$ )
18:    end if
19:  end for
20: end while
21: return None

```

3.6 Path Reconstruction

When the goal state is reached, the path is recovered by following parent pointers from the goal node back to the start, then reversing:

```

1 path = []
2 node = current           # current is the goal node
3 while node is not None:
4     path.append((node.row, node.col))
5     node = node.parent
6 path.reverse()           # path now runs start -> goal
7 return path

```

Listing 3.1: Path reconstruction from goal node

3.7 Complexity Analysis

Parameter	Symbol	Bound
State space size	$ S $	$O(R \cdot C \cdot T_{\max})$
Open heap operations	—	$O(S \log S)$
Space complexity	—	$O(R \cdot C \cdot T_{\max})$
Completeness	—	Complete (if path exists within $T_{\max}$)
Optimality	—	Optimal (admissible h)

Table 3.1: Space-Time A* complexity

In practice, $T_{\max} = 200$ is used as a safety bound. Paths longer than 200 steps indicate a configuration where no plan is feasible.

Chapter 4

Algorithm 1: Independent A* (Baseline)

4.1 Concept

Independent A* is the simplest possible multi-agent strategy: plan each robot's path *as if it were the only robot in the warehouse*, ignoring all other agents. Conflicts are detected after all paths are planned, and a simple wait-and-retry heuristic attempts to patch them.

4.2 Algorithm

Algorithm 3 Independent A*

Require: Task $\mathcal{T}$ with N robots

```
1:  $\mathcal{R} \leftarrow \emptyset$  ▷ Empty reservation table
2: for  $i = 1$  to  $N$  do
3:    $\pi_i \leftarrow \text{SPACE\_TIME\_A\_STAR}(G, s_i, g_i, \mathcal{R})$  ▷ Ignores other robots
4: end for
5: for retry = 1 to MAX_RETRIES do
6:    $C \leftarrow \text{DETECT\_CONFLICTS}(\pi_1, \dots, \pi_N)$ 
7:   if  $C = \emptyset$  then break
8:   end if
9:   for each conflict  $c \in C$  do
10:     $v \leftarrow$  lower-priority robot in  $c$ 
11:    Insert WAIT step at conflict time in  $\pi_v$ 
12:   end for
13: end for
14: return  $\pi_1, \dots, \pi_N$ 
```

4.3 Why It Fails in Dense Environments

Important

Independent A* is **incomplete** and **non-optimal** for MAPF:

- Inserting a WAIT step for one robot can create a new conflict with a third robot, causing cascading conflicts.
- In narrow corridors, two robots may block each other indefinitely — a **deadlock** — that wait-and-retry cannot resolve.
- Without a global view, the retry loop may cycle without converging.

4.4 Use Case

Independent A* is used exclusively as a **baseline** to quantify the improvement provided by cooperative methods. Its success rate degrades sharply as density increases (number of robots / number of free cells).

Chapter 5

Algorithm 2: Cooperative A* (CA*)

5.1 Concept

Cooperative A* (CA*) is a *hierarchical* or *sequential priority* planning algorithm. Robots are assigned a fixed priority order. The highest-priority robot plans first with no constraints. Every subsequent robot plans using Space-Time A* with the reservation table populated by all previously planned robots.

Key Insight

By construction, each new robot's path *cannot conflict* with any already-planned path, because the reservation table blocks all those positions. This makes CA* **conflict-free by design** — no post-hoc checking needed.

5.2 Algorithm

Algorithm 4 Cooperative A* (CA*)

Require: Task $\mathcal{T}$, priority order σ

```
1:  $\mathcal{R} \leftarrow \emptyset$  ▷ Empty, shared reservation table
2: for  $i = 1$  to  $N$  (in priority order  $\sigma$ ) do
3:    $\pi_i \leftarrow \text{SPACE\_TIME\_A\_STAR}(G, s_i, g_i, \mathcal{R})$ 
4:   if  $\pi_i \neq \text{None}$  then
5:      $\text{RESERVE}(\mathcal{R}, \pi_i, i)$  ▷ All future robots avoid robot  $i$ 's path
6:   else
7:     Report failure for robot  $i$ 
8:   end if
9: end for
10: return  $\pi_1, \dots, \pi_N$ 
```

5.3 Detailed Walk-through

Consider 3 robots with priority order $[R_0, R_1, R_2]$:

1. R_0 plans with $\mathcal{R} = \emptyset$. Gets shortest path. Reservations for R_0 's cells are added to $\mathcal{R}$.

2. R_1 plans with $\mathcal{R}$ containing R_0 's reservations. Space-Time A* naturally routes R_1 around R_0 , possibly adding WAIT steps if a shared corridor is temporarily blocked. R_1 's reservations are added to $\mathcal{R}$.
3. R_2 plans with $\mathcal{R}$ containing both R_0 and R_1 . R_2 gets the most constrained (and potentially longest) path.

5.4 Properties

Property	Details
Completeness	Complete for each robot individually, given the current reservations. Globally complete if the grid is not over-congested.
Optimality	Not globally optimal. Earlier robots get optimal paths; later robots may take long detours.
Conflict-free	Guaranteed by construction — no post-hoc conflict checking needed.
Priority dependence	Different priority orderings yield different (sometimes significantly different) total costs.
Planning time	$O(N \cdot T_{\max} \cdot RC \log(RC \cdot T_{\max}))$

Table 5.1: Cooperative A* properties

5.5 Priority Ordering Strategies

The choice of priority order significantly affects solution quality:

- **Arbitrary** (default): assign ID order. Simple but suboptimal.
- **Shortest-path-first**: plan robots with shorter individual paths first — they are less likely to block others.
- **Distance-to-goal**: higher priority for robots farthest from their goals — they need more space to maneuver.
- **Random**: randomize priority order and take the best of several runs.

5.6 Head-on Collision Example

Scenario: two robots face each other in a corridor.

- R_0 : $(0, 0) \rightarrow (0, 7)$ — moving right
- R_1 : $(0, 7) \rightarrow (0, 0)$ — moving left

R_0 plans first (ignoring R_1) and gets the direct path. $\mathcal{R}$ now contains all of R_0 's timestep-positions. When R_1 plans, Space-Time A* discovers that the path R_0 will occupy is blocked at every timestep. R_1 finds a detour (via a parallel row or waiting until R_0 has passed), fully resolving the head-on conflict.

Chapter 6

Algorithm 3: Hill Climbing Optimizer

6.1 Motivation

Cooperative A* guarantees conflict-free paths but assigns them in a fixed priority order, leading to suboptimal (longer-than-necessary) paths for lower-priority robots. Hill Climbing is a **local search** post-processor that iteratively improves an already-valid solution.

6.2 Concept

Hill Climbing (Local Search)

A local search algorithm that starts from a complete solution and repeatedly applies small changes (*moves*) to neighboring solutions, accepting a move only if it strictly improves the objective value. It terminates when no improving move exists (local optimum) or a maximum iteration count is reached.

In our context:

- **Current solution:** all N robot paths from CA*
- **Neighbor move:** re-plan one robot i with an empty slot in the reservation table (remove i 's path and try to find a shorter one)
- **Objective:** minimize total flowtime $\sum_i |\pi_i|$

6.3 Algorithm

Algorithm 5 Hill Climbing Path Optimizer

Require: Valid CA* solution $\Pi = \{\pi_1, \dots, \pi_N\}$, MAX_ITER

```

1: improved  $\leftarrow$  True, iter  $\leftarrow$  0
2: while improved and iter < MAX_ITER do
3:   improved  $\leftarrow$  False
4:   iter += 1
5:   for  $i = 1$  to  $N$  do
6:     Build  $\mathcal{R}_{-i}$ : reserve all paths except  $\pi_i$ 
7:      $\pi'_i \leftarrow$  SPACETIMESTAR( $G, s_i, g_i, \mathcal{R}_{-i}$ )
8:     if  $|\pi'_i| < |\pi_i|$  then
9:        $\pi_i \leftarrow \pi'_i$  ▷ Accept improvement
10:    improved  $\leftarrow$  True
11:  end if
12: end for
13: end while
14: return  $\Pi$ 

```

6.4 Why Conflict-Freedom is Preserved

After removing π_i from $\mathcal{R}$, the new path π'_i is planned with $\mathcal{R}_{-i}$ — which still contains all other robots' paths. Therefore, π'_i cannot conflict with any other robot's path. The solution remains conflict-free after every accepted improvement.

6.5 Convergence & Limitations

- **Termination:** guaranteed within MAX_ITER iterations, or earlier at a local optimum.
- **Local optimum:** Hill Climbing can get stuck. For example, if shortening robot i 's path would require first lengthening robot j 's path (which Hill Climbing will not do), the global optimum may be missed.
- **Typical improvement:** 5–20% reduction in flowtime on top of CA*.

Chapter 7

Algorithm 4: Conflict-Based Search (CBS)

7.1 Overview

Conflict-Based Search (CBS) is a complete, optimal MAPF algorithm introduced by Sharon et al. (2015). It is structured as a two-level search:

- **High Level:** searches a *Conflict Tree* (CT) whose nodes represent sets of constraints
- **Low Level:** re-plans individual agents' paths to satisfy their assigned constraints

CBS is guaranteed to find the **optimal** (minimum-flowtime) conflict-free solution.

7.2 Key Data Structures

7.2.1 CBSConstraint

```
1 class CBSConstraint:
2     """
3     "Robot robot_id must NOT be at position pos at timestep t
4     .
5     """
6     def __init__(self, robot_id, pos, t):
7         self.robot_id = robot_id
8         self.pos      = pos      # (row, col)
9         self.t        = t        # timestep
```

Listing 7.1: A single CBS constraint

7.2.2 CBSNode

Each node in the Conflict Tree stores:

- A set of constraints (accumulated from the root)
- Paths for all robots satisfying those constraints
- A cost value (sum of path lengths) for heap ordering

7.3 High-Level Algorithm

Algorithm 6 CBS High Level — Conflict Tree Search

Require: Task $\mathcal{T}$

```

1: Root: no constraints; plan each robot independently
2: OPEN  $\leftarrow$  {Root} ▷ Min-heap by cost
3: while OPEN  $\neq \emptyset$  do
4:    $n \leftarrow$  HEAPPOP(OPEN)
5:    $c \leftarrow$  FINDFIRSTCONFLICT( $n$ .paths)
6:   if  $c = \text{None}$  then
7:     return  $n$ .paths ▷ Optimal solution found
8:   end if
9:    $(A, B, \text{pos}, t) \leftarrow c$  ▷ Robots  $A$  and  $B$  conflict at pos at time  $t$ 
10:  for robot  $x \in \{A, B\}$  do
11:     $n' \leftarrow$  copy of  $n$ 
12:    Add constraint  $(x, \text{pos}, t)$  to  $n'$ 
13:    Re-plan  $x$ 's path respecting  $n'$ 's constraints
14:    if re-planning succeeds then
15:      HEAPPUSH(OPEN,  $n'$ )
16:    end if
17:  end for
18: end while
19: return None ▷ No solution

```

7.4 Low-Level Algorithm

The CBS low-level planner converts the current robot's constraints into vertex reservations in a temporary reservation table, then calls Space-Time A*:

```

1 def cbs_low_level(grid, robot, constraints):
2     rt = ReservationTable()
3     for c in constraints:
4         if c.robot_id == robot.robot_id:
5             # Forbidden cell at timestep c.t
6             rt.vertex_reservations.add((c.pos[0], c.pos[1], c
7             .t))
8     return space_time_astar(grid, robot.start, robot.goal, rt
9     )

```

Listing 7.2: CBS low-level planner

7.5 Optimality Proof Sketch

Theorem 1. *CBS always finds a solution with minimum flowtime, if one exists.*

Sketch. The Conflict Tree is searched by increasing cost (min-heap). The root node plans all agents independently (minimum individual costs). Every branching step adds

a constraint, which can only increase or maintain individual path costs. The first leaf node with no conflicts has minimum total cost among all valid solutions explored, because all lower-cost nodes were already expanded (and found to have conflicts). By the argument that every conflict-free solution corresponds to at least one CT leaf node, CBS finds the optimal solution. $\square$ $\square$

7.6 Complexity

Measure	Bound
CT nodes (worst case)	Exponential in N
Low-level calls per node	$O(N)$
Total time (worst case)	$O(2^N \cdot N \cdot A^*\text{-cost})$
Practical upper limit	$N \leq 10$ with <code>max_nodes=500</code>

Table 7.1: CBS complexity

Due to exponential growth, CBS becomes impractical for $N > 10$ without enhancements (ECBS, ICTS, LaCAM). For this project, a node expansion limit of 500 is used as a safety bound.

Chapter 8

Visualization System

8.1 Animated Simulation

The `animate_solution` function uses Matplotlib’s `FuncAnimation` to create a frame-by-frame animation:

- Each **frame** corresponds to one timestep $t \in \{0, \dots, \text{makespan}\}$
- Each robot is drawn as a filled circle at its position $\pi_i(t)$, labeled with its ID
- Each robot’s goal is drawn as a dashed rectangle in the robot’s color
- The timestep counter is displayed in the title

8.2 Congestion Heatmap

$$H[r][c] = \sum_{i=1}^N \sum_{t=0}^{|\pi_i|} \mathbf{1}[\pi_i(t) = (r, c)] \quad (8.1)$$

$H[r][c]$ counts the total number of (robot, timestep) pairs that place a robot at cell (r, c) . This directly measures corridor usage and identifies bottlenecks. The heatmap is rendered using Matplotlib’s `imshow` with the `YlOrRd` colormap (yellow = low, red = high).

8.3 Static Snapshot

The `show_static_snapshot` function renders a single timestep with full path trails drawn as faint lines behind each robot — useful for visualizing the complete trajectory layout in one image.

Chapter 9

Comparative Evaluation

9.1 Experiment 1: Success Rate

For each algorithm, 8–10 random MAPF instances are generated on the small warehouse. An instance is *successful* if the algorithm produces paths with zero vertex and zero edge conflicts.

Algorithm	Success Rate	Avg Makespan	Avg Flowtime	Avg Time (s)
Independent A*	~50%	High (varies)	High	Very fast
Cooperative A*	~95%	Moderate	Moderate	Fast
CA* + Hill Climbing	~95%	Lower	Lower	Fast
CBS	~100%	Optimal	Optimal	Slow

Table 9.1: Algorithm comparison (illustrative results)

9.2 Experiment 2: Fleet Size Scaling

Cooperative A* is tested with increasing fleet sizes $N \in \{3, 5, 8, 10, 12, 15\}$ on the medium warehouse. Expected trends:

- Makespan grows roughly linearly with N (more traffic = longer waits)
- Flowtime grows faster than linearly (later robots take longer detours)
- Planning time grows linearly in N (each robot's A* is independent)

9.3 Experiment 3: Head-on Collision

Two robots are placed facing each other in a narrow corridor. Independent A* will always produce a swap conflict. CA* gracefully resolves it: the lower-priority robot's planner detects that the direct path is blocked (edge-reserved) and finds a detour or waits, producing zero conflicts.

Chapter 10

Summary & Conclusions

10.1 Algorithm Comparison Table

Algorithm	Optimal?	Complete?	Speed	Best Case	Use
Independent A*	No	No (deadlocks possible)	Very fast	Sparse maps, prototype	
Cooperative A*	No	Yes (if paths exist)	Fast	Production baseline	
CA* + Hill Climbing	No	Yes	Fast	Improving CA* quality	
CBS	Yes	Yes	Slow (≤ 10 robots)	Small optimal problems	

Table 10.1: Summary comparison of all implemented algorithms

10.2 Key Findings

1. **Reservation Tables** with $O(1)$ lookup are the key enabling data structure. They allow any algorithm to check conflict-freedom at any (r, c, t) triple in constant time.
2. **Space-Time A*** naturally handles dynamic obstacles by searching in 3D space. The WAIT action is discovered automatically rather than being inserted ad hoc.
3. **Cooperative A*** eliminates conflicts by construction and scales to large fleets efficiently ($O(N)$ planning calls). Its main limitation is priority-order sensitivity.
4. **Hill Climbing** consistently reduces flowtime by 5–20% over CA* with minimal computational overhead.
5. **CBS** is the only algorithm guaranteed to find the optimal solution, but its exponential worst-case growth limits it to $N \leq 10$ in practice.

6. **Head-on collisions** are handled naturally by the edge reservation mechanism — no special-case code is required.

10.3 Limitations & Future Work

- **Lifelong MAPF**: the current system handles one-shot tasks. In real warehouses, tasks appear continuously; robots must be re-routed online.
- **Priority optimization**: better priority orderings (e.g., based on path length or urgency) would improve CA* quality.
- **Enhanced CBS**: ECBS (Enhanced CBS with focal search) or LaCAM provide better scalability while retaining near-optimality.
- **Real benchmark maps**: the MovingAI warehouse benchmark maps (<https://movingai.com/benchmarks/>) provide standardized large-scale test cases.
- **Diagonal movement**: extending the action space to 8 directions allows shorter paths in open spaces.
- **Multi-objective optimization**: simultaneously minimizing makespan, flowtime, and energy consumption.

Appendix A

Class & Function Reference

Class / Function	Section	Description
WarehouseGrid	§2.2	2D grid environment
ReservationTable	§2.3	Space-time occupancy tracker
SpaceTimeNode	§3	Search node for ST-A*
space_time_astar	§3	Core 3D A* planner
manhattan_distance	§3.3	Admissible heuristic
Robot	§2.4	Single robot entity
WarehouseTask	§2.4	MAPF problem instance
generate_random_task	§2.4	Random task generator
plan_independent_astar	§4	Baseline planner
plan_cooperative_astar	§5	CA* planner
optimize_hill_climbing	§6	Post-optimizer
CBSConstraint	§7.2	CBS prohibition constraint
CBSNode	§7.2	Conflict tree node
cbs_low_level	§7.3	CBS individual re-planner
find_first_conflict	§7	CT conflict detector
plan_cbs	§7	Full CBS planner
animate_solution	§8.1	Matplotlib animation
show_static_snapshot	§8.3	Single-frame renderer
plot_congestion_heatmap	§8.2	Corridor usage heatmap
compare_algorithms	§9	Success rate comparison
fleet_size_scaling	§9	Scalability benchmarking
results_dashboard	§9	Full comparison dashboard

Table A.1: Complete class and function reference

Appendix B

Conflict Type Reference

Conflict Type	Definition	Prevention Mechanism
Vertex conflict	$\pi_i(t) = \pi_j(t)$ for $i \neq j$	Vertex reservation in $\mathcal{R}_v$
Edge (swap) conflict	$\pi_i(t) = \pi_j(t - 1)$ and $\pi_i(t - 1) = \pi_j(t)$	Edge reservation in $\mathcal{R}_e$ (both directions)
Follow conflict	Robot j enters a cell immediately after robot i leaves	Handled implicitly by vertex reservations with tail extension

Table B.1: Conflict types and prevention mechanisms

Appendix C

Glossary

MAPF	Multi-Agent Pathfinding — finding conflict-free paths for N agents
Makespan	Time until the last robot reaches its goal ($\max \pi_i $)
Flowtime	Sum of all path lengths ($\sum \pi_i $)
Reservation Table	Data structure mapping $(r, c, t) \rightarrow$ occupied
Space-Time A*	A* operating in the $(r, c, \text{timestep})$ state space
Vertex Conflict	Two agents at the same cell at the same time
Edge Conflict	Two agents swapping positions in one step
Conflict Tree	CBS's high-level search structure (a binary tree of constraint sets)
Admissible Heuristic	A heuristic that never overestimates true cost
Cooperative A*	Sequential planning where later robots treat earlier paths as dynamic obstacles
Hill Climbing	Local search that improves one robot's path at a time
CBS	Conflict-Based Search — optimal two-level MAPF algorithm
Deadlock	A state where no robot can make progress (cycle of blockages)

Bibliography

- [1] G. Sharon, R. Stern, A. Felner, N. R. Sturtevant, *Conflict-based search for optimal multi-agent pathfinding*, Artificial Intelligence, 219:40–66, 2015.
- [2] D. Silver, *Cooperative pathfinding*, AIIDE 2005.
- [3] N. R. Sturtevant, *Benchmarks for Grid-Based Pathfinding*, IEEE Transactions on Computational Intelligence and AI in Games, 2012. <https://movingai.com/benchmarks/>
- [4] R. Stern et al., *Multi-Agent Pathfinding: Definitions, Variants, and Benchmarks*, SoCS 2019.